



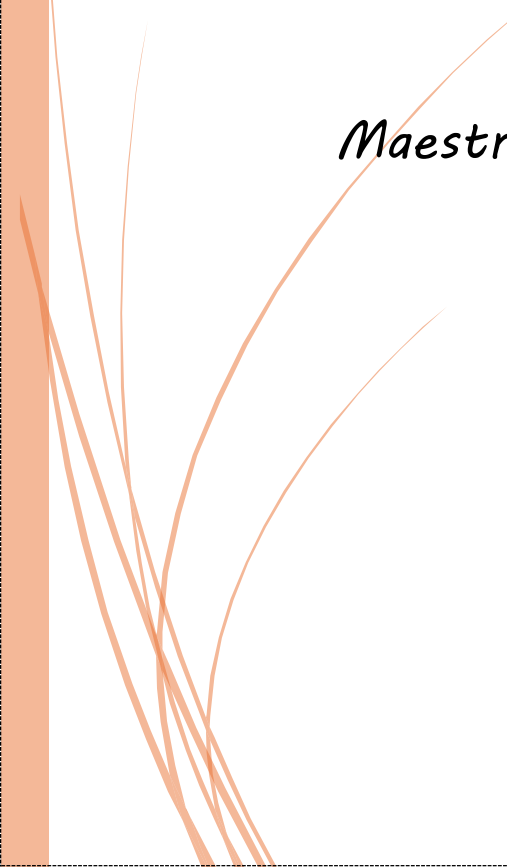
Universidad Tecnológica De Nezahualcóyotl

*Documentación Chat IA, Token, Verificación en 2
pasos y Pagos*

Alumno(a): Solis Garcia Fany Sarai

Materia: Aplicaciones Web Orientada a Objetos

Maestro: Del Prado López Jovan



1 Estructura General de la Aplicación

¿Qué hace esta aplicación?

Es una **tienda de dulces mexicanos con flujo de compra completo**. Presenta un catálogo de 30 productos en cuadrícula, permite agregar artículos a un carrito lateral deslizante, elegir el método de pago y finalizar la compra mediante una **verificación en dos pasos (2FA)**: el usuario escanea un código QR e introduce un PIN de 6 dígitos. Al confirmar se muestra un comprobante bancario simulado con número de folio. Además cuenta con un **chat flotante impulsado por IA** (Grog) con reglas de comportamiento estrictas.

Árbol de bloques principales del HTML

Bloque / ID o Clase	Tipo	Propósito
header	Fijo	Logo circular del gato, título y slógan. Borde inferior rojo como acento de marca
button.cart-trigger	Flotante	Botón fijo en esquina superior derecha; muestra el conteo de ítems del carrito
button.chat-trigger	Flotante	Botón circular fijo en esquina inferior derecha; abre y cierra el chat con IA
div#sidebar	Panel lat.	Carrito deslizante oculto (right:-400px); contiene lista, métodos de pago y botón de pago
div.container > div.grid	Cuadrícula	Grid CSS auto-fill con las 30 tarjetas de productos generadas por JavaScript
div#modal-2fa	Modal	Pantalla de verificación en dos pasos con código QR y campo PIN; oculto hasta pagar
div#chat-window	Chat flot.	Ventana de chat con el asistente IA; oculta por defecto, se activa con el botón gato
form#chat-form	Input	Campo de texto y botón de envío dentro del chat flotante

2 Variables CSS y Estilos Visuales

Variables :root — Paleta centralizada

```
:root { --rojo: #d32f2f; /* Rojo principal: botones, bordes, header */ --crema: #fff5f2; /*  
Fondo de página cálido */ --oscuro: #212121; /* Negro casi puro: logo y texto  
fuerte */ --verde: #2e7d32; /* Verde: estado exitoso del pago */ }
```

Al centralizar los colores en :root, cambiar la identidad visual de toda la aplicación requiere modificar únicamente estas 4 líneas. Se usan en todo el CSS con la función var(--nombre).

Botones flotantes — Posición fija en pantalla

```
.btn-flotante { position: fixed; z-index: 1000; border-radius: 50px; } .cart-trigger { top: 20px;  
right: 20px; background: var(--rojo); } .chat-trigger { bottom: 20px; right: 20px; background:  
var(--oscuro); width: 60px; height: 60px; border-radius: 50%; }
```

position:fixed hace que ambos botones permanezcan visibles al hacer scroll. z-index:1000 los mantiene por encima del contenido pero por debajo del sidebar (z-index:1005) y del modal 2FA (z-index:2000). El botón del carrito es una píldora (border-radius:50px); el del chat es un círculo perfecto (border-radius:50%).

#sidebar — Carrito deslizante

```
#sidebar { position: fixed; right: -400px; top: 0; width: 350px; height: 100%; z-index:  
1005; transition: 0.4s; overflow-y: auto; } #sidebar.open { right: 0; }
```

El sidebar comienza fuera de pantalla (right:-400px). Cuando JavaScript agrega la clase 'open', right pasa a 0 y transition:0.4s crea la animación de deslizamiento suave. overflow-y:auto permite hacer scroll dentro del carrito si hay muchos productos.

#modal-2fa — Pantalla de verificación 2FA

```
#modal-2fa { display: none; position: fixed; top: 0; left: 0; width: 100%; height: 100%;  
background: rgba(0,0,0,0.85); z-index: 2000; justify-content: center; align-items: center; }
```

Cubre toda la pantalla con fondo negro semitransparente (0.85 de opacidad) para bloquear la interacción con el resto de la página durante la verificación. z-index:2000 lo coloca por encima de todo. display:flex centra el cuadro blanco vertical y horizontalmente.

#chat-window y burbujas de mensajes

```
#chat-window { position: fixed; bottom: 90px; right: 20px; width: 320px; height: 450px;  
display: none; flex-direction: column; z-index: 1001; } #chat-window.active { display: flex; }
```

```
.msg-u { align-self: flex-end; background: var(--rojo); color: white; } .msg-a { align-self: flex-start; background: #eee; }
```

La ventana se posiciona sobre el botón del gato (bottom:90px). display:flex en columna hace que el historial de mensajes ocupe todo el espacio y el formulario quede anclado abajo. align-self posiciona las burbujas: el usuario a la derecha (rojo) y el asistente a la izquierda (gris), igual que en WhatsApp.

3 Catálogo de Productos

Estructura del array de dulces

Los 30 productos se almacenan en un array de objetos JavaScript. Cada objeto usa propiedades cortas: **n** = nombre, **s** = precio por pieza, **b** = precio por bolsa.

```
const dulces = [ { n: 'Mazapán De la Rosa', s: 6, b: 130 }, { n: 'Pelón Pelo Rico', s: 12, b: 110 }, // ... 28 dulces más ];
```

Renderizado dinámico con forEach

Al cargar la página, un `forEach` recorre el array y construye el HTML de cada tarjeta con `template literals`, evitando escribir 30 bloques manualmente:

```
dulces.forEach((d, i) => { g.innerHTML += `<div class='card'><b>${d.n}</b><br><p>${d.s} / ${d.b}</p><select id='m-${i}'>...</select><button onclick='add(${i})'>Añadir</button></div>`; });
```

⚠ Cada `select` recibe un ID único (`m-0`, `m-1`, `m-2...`) basado en el índice. Esto permite que la función `add(idx)` lea exactamente el modo de compra del producto que se está agregando.

Inventario completo — 30 dulces

#	Dulce	Pieza	Bolsa
1	Mazapán De la Rosa	\$6	\$130
2	Pelón Pelo Rico	\$12	\$110
3	Pulparindo	\$5	\$85
4	Alegría Amaranto	\$15	\$140
5	Borrachitos	\$2	\$45
6	Rockaleta	\$8	\$150
7	Obleas con Cajeta	\$18	\$160
8	Palanqueta Nuez	\$22	\$200
9	Cocada Blanca	\$18	\$170
10	Manzana Chamoy	\$35	\$300
11	Ate de Membrillo	\$40	\$380
12	Puerquito de Pan	\$12	\$110
13	Duvalín Bisabor	\$7	\$120

14	Chicles Canel's	\$2	\$90
15	Pica Fresa	\$1	\$70
16	Skwinkles Rellenos	\$14	\$135
17	Lucas Muecas	\$15	\$145
18	Glorias Linares	\$12	\$210
19	Jamoncillo Leche	\$25	\$230
20	Camote Puebla	\$20	\$180
21	Muégano Tradicional	\$10	\$95
22	Limón con Coco	\$15	\$140
23	Chocomelvavisco	\$10	\$180
24	Barra Chocolate	\$22	\$350
25	Vaso de Cajeta	\$8	\$150
26	Piruleta Rayada	\$6	\$110
27	Galleta Sol	\$5	\$90
28	Morelianas	\$15	\$140
29	Tama-Roca	\$10	\$95
30	Bandera de Coco	\$15	\$140

add(idx) — Agregar producto al carrito

```
function add(idx) {  const m = document.getElementById('m-' + idx).value;  cart.push({  
  id: Date.now(),    // ID único basado en timestamp    n: dulces[idx].n,    p: m === 'b' ?  
  dulces[idx].b : dulces[idx].s  });  upd(); }
```

Lee el valor del select del producto (s=pieza, b=bolsa) y usa el operador ternario para asignar el precio correcto. Cada llamada siempre agrega un ítem nuevo. Usa Date.now() como ID único para poder eliminarlo con precisión.

del(id) — Eliminar ítem por ID único

```
function del(id) {  cart = cart.filter(x => x.id !== id);  upd(); }
```

filter() crea un nuevo array con todos los ítems cuyo id sea distinto al que se quiere eliminar. Usar Date.now() como ID garantiza que solo se elimina ese ítem aunque haya otros del mismo dulce.

upd() — Actualizar vista del carrito

```
function upd() {  let t = 0; l.innerHTML = "";  cart.forEach(x => {    t += x.p;  
    l.innerHTML += `<div class='item-carrito'>      ${x.n} ($${x.p})      <button  
    onclick='del(${x.id})'>icono</button>    </div>`;  });  document.getElementById('cart-  
count').innerText = cart.length;  document.getElementById('total-precio').innerText = '$' +  
t.toFixed(2); }
```

Limpia y reconstruye la lista completa en cada cambio. Suma los precios de todos los ítems para calcular el total. Actualiza el contador del botón flotante. toFixed(2) garantiza que el total siempre muestre dos decimales.

selMet() — Seleccionar método de pago

```
function selMet(m, el) {  document.querySelectorAll('.metodo-pago')    .forEach(x =>  
x.classList.remove('selected'));  el.classList.add('selected');  met = m;  
document.getElementById('btn-comprar').style.display = 'block'; }
```

Quita la clase 'selected' de todos los métodos y la agrega al que se eligió, garantizando que solo uno esté activo a la vez. Guarda el nombre del método en la variable global met. El botón PAGAR AHORA permanece oculto hasta que se elija un método.

5 Token de Verificación en Dos Pasos (2FA)

¿Qué es la verificación en dos pasos?

La **verificación en dos pasos (2FA)** es una capa de seguridad que exige al usuario demostrar su identidad con dos elementos independientes antes de autorizar el pago. El primer factor es elegir el método de pago; el segundo es introducir un **código de un solo uso (OTP)** generado por una aplicación como Google Authenticator o Authy. Esto impide que un tercero complete una compra aunque conozca los datos de la tarjeta, porque necesitaría también el teléfono del usuario.

¿Cómo funciona el código QR?

La librería **qrcodejs** convierte la cadena `otpauth://` en una imagen QR. Al escanearla con Google Authenticator, la app queda enlazada y genera códigos de 6 dígitos cada 30 segundos:

```
function abrir2FA() { document.getElementById('qrcode').innerHTML = ""; // limpia QR anterior
  new QRCode(document.getElementById('qrcode'), { text: 'otpauth://totp/Siete?secret=KVKFKRJTMRXS4BQ', width: 120, height: 120 });
  document.getElementById('modal-2fa').style.display = 'flex'; }
```

Desglose de la URL `otpauth://`

Parte del token	Valor de ejemplo	Cómo se genera
Protocolo	<code>otpauth://</code>	Indica a la app de autenticación que es una cuenta OTP estándar
Tipo	<code>totp</code>	Time-based OTP: el código cambia automáticamente cada 30 segundos
Nombre de cuenta	Siete	Texto que aparece como etiqueta en la app de autenticación del usuario
Parámetro <code>secret=</code>	KVKFKRJTMRXS4BQ	Clave secreta en Base32 compartida entre el servidor y la app del usuario

Función `procesarPagoDefinitivo()` — 6 pasos

```
function procesarPagoDefinitivo() { const pin = document.getElementById('pin-input').value;
  if (pin.length < 6) return alert('Escribe el código de 6 dígitos.');
```

```
  const total = document.getElementById('total-precio').innerText;
  document.getElementById('modal-dinamico').innerHTML = `
    <h2>PAGO AUTORIZADO</h2>
    Folio: SV-${Math.floor(Math.random()*899999 + 100000)}
    Método: ${met}
    Monto: ${total}
    Estado: EXITOSO
    <button onclick='location.reload()'>ACEPTAR</button>`;
  cart = [];
  upd(); }
```


Paso	Descripción
1 — Validación de longitud	Si el PIN tiene menos de 6 dígitos, muestra una alerta y detiene la ejecución. No procede al pago.
2 — Lectura del total	Lee el texto del elemento #total-precio para incluirlo en el comprobante final.
3 — Generación del folio	Math.random()*899999+100000 genera un número entre 100000 y 999999. Math.floor() lo convierte en entero de 6 dígitos.
4 — Sustitución del modal	innerHTML del modal-dinámico se reemplaza completamente: desaparecen el QR y el PIN y aparecen el ícono de éxito y el comprobante.
5 — Vaciado del carrito	cart = [] reinicia el array y upd() actualiza el contador y la lista visual a cero.
6 — Botón ACEPTAR	location.reload() recarga la página completa, reiniciando la aplicación a su estado inicial.

⚠ En un entorno de producción real, la validación del PIN debe realizarse en el servidor usando una librería TOTP (como pyotp en Python o speakeasy en Node.js). En esta simulación, cualquier PIN de 6 dígitos es aceptado.

¿Qué es Groq y cómo se conecta?

Groq es una plataforma de inferencia de IA ultrarrápida. La aplicación se conecta a su endpoint compatible con OpenAI usando `fetch()` de JavaScript. El modelo usado es **llama-3.3-70b-versatile**, un LLM de Meta con 70,000 millones de parámetros que se ejecuta en los chips LPU de Groq con latencia muy baja.

Reglas de comportamiento del asistente

Regla	Comportamiento esperado	Por qué es importante
Regla 1	Solo responde sobre dulces y precios del catálogo	Evita que el asistente se desvíe a temas irrelevantes como política, recetas u otros comercios
Regla 2	Declina amablemente preguntas fuera de tema	Mantiene la experiencia profesional sin respuestas confusas o fuera de contexto
Regla 3	Si el dulce no existe en el catálogo, simula que se agotó y estima un precio	Evita respuestas secas de 'no tengo ese producto' y mantiene al cliente interesado
Catálogo iny.	JSON.stringify(dulces) incrusta los 30 productos en el prompt	El modelo tiene acceso a nombres y precios exactos sin necesidad de una base de datos externa

Evento submit del chat — flujo async/await

```
document.getElementById('chat-form').addEventListener('submit', async (e) => {
  e.preventDefault(); // evita recargar la página
  const val = i.value; i.value = "";
  // lee y limpia el input
  b.innerHTML += '<div class=msg-u>' + val + '</div>';
  const r = await
  fetch('https://api.groq.com/openai/v1/chat/completions', {
    method: 'POST',
    headers: {
      'Authorization': 'Bearer ' + API_KEY,
      'Content-Type': 'application/json'
    },
    body:
    JSON.stringify({
      model: 'llama-3.3-70b-versatile',
      messages: [
        { role: 'system', content: promptContext },
        { role: 'user', content: val }
      ]
    })
  });
  const d = await r.json();
  b.innerHTML += '<div class=msg-a>' +
  d.choices[0].message.content + '</div>';
  b.scrollTop = b.scrollHeight; // auto-scroll al último
  mensaje });
```

Parámetros de la llamada a la API

Parámetro	Valor	Descripción
-----------	-------	-------------

method	POST	Las peticiones a la API siempre son POST con el cuerpo en JSON
Authorization	Bearer [API_KEY]	Autenticación tipo Bearer Token requerida por Groq
model	llama-3.3-70b-versatile	Modelo LLM de Meta hospedado en Groq; muy capaz y rápido
messages[system]	Prompt del sistema	Define la personalidad y contexto del asistente (nombre, catálogo, reglas)
messages[user]	Mensaje del usuario	Lo que el usuario escribió en el input del chat
choices[0].message	Respuesta del modelo	Texto de respuesta que se agrega al chat-box como burbuja del asistente

⚠ e.preventDefault() es esencial: sin él, el form enviaría la página y recargara el navegador, perdiendo el historial del chat y el carrito completo. b.scrollTop = b.scrollHeight desplaza el historial al último mensaje automáticamente.

7 Flujo Completo de una Compra con 2FA

#	Actor	Acción
1	Navegador	Carga la página, aplica CSS e importa la librería qrcodejs
2	JavaScript	forEach renderiza las 30 tarjetas de dulces en el grid
3	Usuario	Selecciona modo (Pieza/Bolsa) y pulsa Añadir en un dulce
4	add()	Crea un ítem con ID único (Date.now()) y lo agrega al array cart
5	upd()	Actualiza el contador del botón flotante y la lista del carrito
6	Usuario	Pulsa el botón flotante 'Mi Pedido' para abrir el sidebar
7	toggleSidebar()	Agrega la clase 'open' al sidebar: se desliza desde la derecha
8	Usuario	Elige un método de pago (Mercado Pago, Apple Pay o PayPal)
9	selMet()	Marca el método como 'selected' y hace visible el botón PAGAR AHORA
10	Usuario	Pulsa PAGAR AHORA
11	abrir2FA()	Genera el código QR con qrcodejs y muestra el modal de verificación 2FA
12	Usuario	Escanea el QR con Google Authenticator u otra app de autenticación
13	App autenticación	Genera un código OTP de 6 dígitos válido por 30 segundos
14	Usuario	Introduce el PIN de 6 dígitos en el campo del modal
15	procesarPago()	Valida que tenga 6 dígitos, genera el folio y reemplaza el modal con el comprobante
16	Pantalla	Muestra: PAGO AUTORIZADO, folio, método, monto y estado EXITOSO
17	Usuario	Pulsa ACEPTAR: location.reload() reinicia la aplicación

⚠ Los pasos 11–15 (fondo naranja claro) corresponden al flujo de verificación en dos pasos.

Resumen de tecnologías utilizadas

Tecnología / Patrón	Uso en el proyecto
HTML5 semántico	header, section, div, form, input, button organizados en bloques funcionales

CSS Variables :root	Paleta de 4 colores centralizada: rojo, crema, oscuro y verde
CSS Grid auto-fill	Cuadrícula de tarjetas completamente responsiva sin media queries
CSS Flexbox	Burbujas de chat alineadas, botones centrados, layout del sidebar
position:fixed + transition	Botones flotantes siempre visibles; sidebar y chat con animación deslizante
JavaScript Vanilla	forEach, filter, template literals, classList, Date.now() — sin frameworks
qrcodejs (librería CDN)	Convierte la URL otpauth:// en imagen de código QR dentro del modal
Protocolo TOTP / otpauth	Estándar RFC 6238 para códigos de un solo uso compatibles con Google Authenticator
Fetch API async/await	Llamada asíncrona a Groq sin bloquear la interfaz mientras espera respuesta
Groq API + LLaMA 3.3	Chat IA con reglas de comportamiento inyectadas en el prompt del sistema
Modal dinámico	El mismo div del 2FA se reutiliza para mostrar el comprobante bancario final